

Detail	Information
Subject	Database Fundamentals
Semester	2nd Semester , Information Systems
Total Questions	4 Questions
Total Points	100 Points
Platform	Oracle APEX (apex.oracle.com)
Duration	120 Minutes
Type	Individual , Open Access Allowed

Company Background

Buku Pintar is a book rental shop located in Semarang, Indonesia. The shop has been operating since 2018 and serves university students, high school students, and general readers who prefer to rent books at an affordable price rather than buying them outright. The shop currently manages two categories of books: academic textbooks and general fiction or non-fiction books. Buku Pintar processes approximately 30 rentals per day and employs 6 staff members in total covering roles as receptionist, cashier, and manager.

The owner, Pak Arief, currently manages all operations using a handwritten logbook and basic spreadsheet. As the business grows and complaints about lost records and overdue books increase, he has decided to build a new Book Rental Management System (BRMS) using Oracle database. The system must handle member registration, book catalog management, rental transactions, and payment recording.

Business Processes

- A customer registers as a member before renting. Each member has a unique membership code, personal details, and a membership type: 'student' or 'general'. Student members receive a 10% discount on all rentals.
- Each book in the catalog belongs to a genre category such as 'Academic', 'Fiction', 'Non-Fiction', or 'Children'. A book has a daily rental price, and the shop tracks how many copies are available in stock.
- When a member wants to rent a book, a staff member records the rental. Each rental covers exactly one book. The system records the rental date, the expected return date, the actual return date (if returned), and the rental status.
- The rental fee is calculated based on daily price multiplied by the number of days rented. If a book is returned late, an additional late fee of Rp 2,000 per day is charged on top of the regular rental fee.
- After returning the book, the cashier records the payment. Payment can be made by cash, transfer, or GoPay. The system records the total amount paid and any change given.

Database Entities , 5 Tables

The Book Rental Management System must be implemented with exactly the following 5 tables:

Table	Key Attributes	Business Rules & Constraints
MEMBER	member_id (PK), member_code, full_name, phone, email, member_type, join_date, is_active	member_code is unique. member_type must be 'student' or 'general'. is_active must be 'Y' or 'N'. join_date defaults to SYSDATE.
BOOK	book_id (PK), isbn, title, author, genre, daily_price, stock_qty, is_available	isbn is unique. genre must be 'Academic', 'Fiction', 'Non-Fiction', or 'Children'. daily_price must be > 0. stock_qty must be >= 0. is_available must be 'Y' or 'N'.
STAFF	staff_id (PK), full_name, email, phone, role, salary, hire_date	email is unique. role must be 'receptionist', 'cashier', or 'manager'. salary must be > 0. hire_date defaults to SYSDATE.
RENTAL	rental_id (PK), member_id (FK), book_id (FK), staff_id (FK), rental_date, expected_return_date, actual_return_date, rental_days, rental_fee, late_fee, rental_status	rental_date defaults to SYSDATE. rental_days must be >= 1. rental_fee and late_fee must be >= 0. rental_status must be 'active', 'returned', or 'overdue'. actual_return_date is optional (NULL while still rented).
PAYMENT	payment_id (PK), rental_id (FK), staff_id (FK), payment_date, amount_paid, payment_method, change_given	rental_id must be UNIQUE (one payment per rental). payment_method must be 'cash', 'transfer', or 'gopay'. amount_paid must be > 0. change_given defaults to 0.

Database Users and Access Rights

User Account	Role	Access Requirements
db_owner	Database Owner	Full access (SELECT, INSERT, UPDATE, DELETE) on ALL tables. Can GRANT to others.
receptionist_user	Receptionist	SELECT on MEMBER, BOOK. INSERT and SELECT on RENTAL. UPDATE stock_qty on BOOK. INSERT on MEMBER.
cashier_user	Cashier	SELECT on RENTAL, MEMBER. INSERT and SELECT on PAYMENT. UPDATE rental_status on RENTAL. UPDATE payment_status column does not exist here , UPDATE rental_status only.
report_user	Management	SELECT ONLY on ALL 5 tables. No INSERT, UPDATE, or DELETE.

QUESTION 1 [30 Points] ERD Design and DDL , CREATE TABLE with Constraints

Task Description

Based on the case study of Buku Pintar Book Rental, complete the following tasks:

Part A , Entity Relationship Diagram (15 Points)

- Draw a complete ERD for the Buku Pintar Book Rental Management System using all 5 entities.
- Show all relationships between entities with correct cardinality using crow's foot notation.
- Clearly label all Primary Keys (PK) and Foreign Keys (FK).
- Show mandatory and optional relationships correctly on both sides.
- Label the data type for each attribute in every entity.

Part B , DDL CREATE TABLE Statements (15 Points)

Write complete Oracle DDL CREATE TABLE statements for Min 3 tables. Your statements must include:

- Entity Integrity: Named PRIMARY KEY constraint using CONSTRAINT keyword for each table
- Referential Integrity: Named FOREIGN KEY constraints referencing the correct parent table and column
- Domain / Check Constraints: At least one named CHECK constraint per table enforcing the business rules
- NOT NULL on all mandatory columns
- DEFAULT values where stated in the case (join_date, rental_date, hire_date, change_given)
- Tables must be created in correct order , parent tables before child tables

QUESTION 2 [25 Points] Database Security (DCL) and Transaction Control (TCL)**Task Description****Part A , GRANT Statements (12 Points)**

Write complete Oracle GRANT statements for each of the following three users based on the access rights defined in the case study:

- receptionist_user
- cashier_user
- report_user

Requirements: Follow the Principle of Least Privilege. Use column-level GRANT where only specific columns need UPDATE access.

Part B , TCL Transaction Scenario (13 Points)

A student member named Farhan Maulana (member_id = 1) rents the book 'Algoritma dan Pemrograman' (book_id = 3) for 5 days. The daily price is Rp 5,000. The receptionist (staff_id = 1) records the rental. Farhan returns the book 2 days late. The cashier (staff_id = 2) processes the payment. Farhan pays Rp 60,000 cash.

Calculation: rental_fee = 5 days x Rp 5,000 = Rp 25,000. late_fee = 2 days x Rp 2,000 = Rp 4,000. Total = Rp 29,000. Farhan pays Rp 60,000, change = Rp 31,000.

Write a complete TCL script that:

- Inserts the RENTAL record with rental_status = 'active'
- Decreases book stock_qty by 1 in BOOK table
- Creates a SAVEPOINT named after_rental
- Updates RENTAL: sets actual_return_date = SYSDATE, late_fee = 4000, rental_status = 'returned'
- Inserts the PAYMENT record
- Increases book stock_qty by 1 (book is back in stock)
- COMMITs if all steps succeed
- Explains ROLLBACK TO SAVEPOINT scenario if the payment step fails

QUESTION 3 [20 Points] Data Manipulation Language , INSERT and UPDATE**Task Description****Part A , INSERT Statements (10 Points)**

Write exactly 1 INSERT statement per table for min 3 tables. Data must be realistic, consistent with the case study, and inserted in correct FK dependency order. Ensure all constraints are respected.

Table	Requirements for Your 1 Row
MEMBER	1 student member. Include member_code, phone, join_date using TO_DATE(). is_active = 'Y'.
BOOK	1 Academic genre book. isbn must be unique. daily_price > 0. stock_qty >= 1.
STAFF	1 receptionist. Include email and hire_date. Salary must be realistic (in Rupiah).
RENTAL	1 active rental linking your member, book, and staff. rental_days >= 1. rental_fee = rental_days x daily_price. late_fee = 0 (not yet returned).
PAYMENT	1 payment for the rental above. Use 'cash' method. amount_paid > rental_fee. change_given = amount_paid - rental_fee.

Part B , UPDATE Statements (10 Points)

Write one UPDATE statement for scenario below (choose min. 2 scenario). Each must use a correct WHERE clause and reflect a real business event:

- BOOK: Update is_available to 'N' for any book whose stock_qty has dropped to 0
- MEMBER: Update is_active to 'N' for a specific member who has not returned a book for more than 30 days
- RENTAL: Update rental_status to 'overdue' for all active rentals where expected_return_date is earlier than today (SYSDATE)
- STAFF: Give a 10% salary raise to all cashier staff
- BOOK: Update the daily_price of a specific book by title, increasing it by Rp 1,000

QUESTION 4 [25 Points] Basic SELECT Queries , Advanced Data Retrieval**Task Description**

Write Oracle SELECT queries to answer each business question below for Buku Pintar.

Case 1 , Rental Revenue Report (9 Points)

Business Question: The manager wants a complete rental report showing all rentals with status 'returned' or 'overdue', including the member's full name and type, the book title and genre, the staff name who recorded the rental, the total charge (rental_fee + late_fee) as 'Total Charge (Rp)', and a column called 'Member Discount' that shows 'Yes - 10%' if the member is a student or 'No' if general. The report must only show rentals where the total charge (rental_fee + late_fee) is greater than Rp 15,000. Sort by total charge descending, then by member full name ascending.

Your query must include:

- JOIN across all 5 tables: RENTAL, MEMBER, BOOK, STAFF, and PAYMENT
- WHERE filtering by rental_status IN and computed total > 15000
- Arithmetic computed column for total charge
- CASE WHEN for the Member Discount column
- ORDER BY with two columns

Case 2 , Book Availability and Genre Analysis (8 Points)

Business Question: The receptionist needs a list of all books that are currently AVAILABLE (is_available = 'Y') AND have a daily_price NOT between Rp 3,000 and Rp 8,000, OR whose genre is 'Academic' regardless of price. Show the book title, author, genre, daily_price, stock_qty, and a computed column 'Weekly Rental Cost' showing the cost to rent for 7 days. Also show a 'Price Level' column: 'Budget' if daily_price < 3000, 'Standard' if between 3000 and 8000, and 'Premium' if above 8000. Sort by genre ascending, then daily_price descending.

Your query must include:

- WHERE using AND + OR with correct parentheses on BOOK table only
- NOT BETWEEN for price filter
- Arithmetic computed column (daily_price x 7) with alias
- CASE WHEN with three conditions for Price Level
- ORDER BY with two columns

Case 3 , Overdue Member and Staff Performance (8 Points)

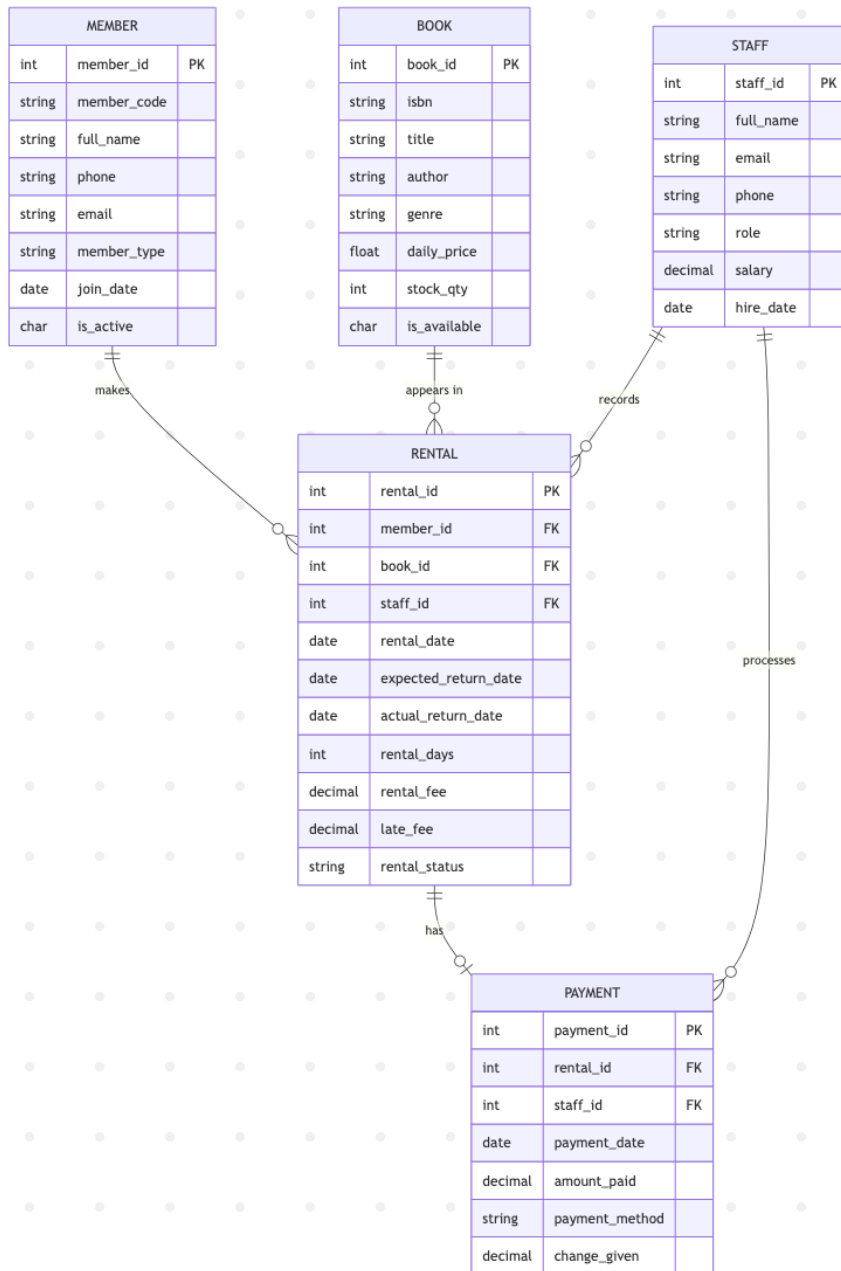
Business Question: The manager needs a report of all ACTIVE or OVERDUE rentals where the member has been renting for more than 7 days (calculated from rental_date to SYSDATE), the rental_fee is above the average rental_fee of all rentals handled by the same staff member (use correlated subquery), and the member type is 'student'. Show: member full name, member_code, book title, staff full name, rental_date formatted as 'DD-MON-YYYY', days rented so far (SYSDATE - rental_date rounded to 0 decimals), rental_fee, and how much the fee exceeds the staff average as 'Above Staff Average (Rp)'. Sort by days rented descending.

Your query must include:

- JOIN across RENTAL, MEMBER, BOOK, and STAFF
- WHERE filtering by rental_status IN, member_type, and date arithmetic (SYSDATE - rental_date > 7)
- Correlated subquery comparing rental_fee to AVG rental_fee of same staff
- TO_CHAR for date formatting and ROUND for days calculation
- Arithmetic column for the above-average difference
- ORDER BY on the computed days column

Solusi :

Q1 - PART A



Q1 - PART B

-- TABLE CREATION ORDER: MEMBER - BOOK - STAFF - RENTAL - PAYMENT

-- 1. MEMBER

```
CREATE TABLE MEMBER (  
  member_id NUMBER GENERATED ALWAYS AS IDENTITY,  
  member_code VARCHAR2(20) NOT NULL,  
  full_name VARCHAR2(150) NOT NULL,  
  phone VARCHAR2(20) NOT NULL,  
  email VARCHAR2(150),  
  member_type VARCHAR2(10) NOT NULL,  
  join_date DATE DEFAULT SYSDATE NOT NULL,  
  is_active CHAR(1) DEFAULT 'Y' NOT NULL,  
  CONSTRAINT pk_member PRIMARY KEY (member_id),  
  CONSTRAINT uq_member_code UNIQUE (member_code),  
  CONSTRAINT chk_member_type CHECK (member_type IN ('student','general')),  
  CONSTRAINT chk_member_actv CHECK (is_active IN ('Y','N'))  
);
```

-- 2. BOOK

```
CREATE TABLE BOOK (  
  book_id NUMBER GENERATED ALWAYS AS IDENTITY,  
  isbn VARCHAR2(20) NOT NULL,  
  title VARCHAR2(255) NOT NULL,  
  author VARCHAR2(150) NOT NULL,  
  genre VARCHAR2(20) NOT NULL,  
  daily_price NUMBER(10,2) NOT NULL,  
  stock_qty NUMBER(5) DEFAULT 0 NOT NULL,  
  is_available CHAR(1) DEFAULT 'Y' NOT NULL,  
  CONSTRAINT pk_book PRIMARY KEY (book_id),  
  CONSTRAINT uq_book_isbn UNIQUE (isbn),  
  CONSTRAINT chk_book_genre CHECK (genre IN ('Academic','Fiction','Non-Fiction','Children')),  
  CONSTRAINT chk_book_price CHECK (daily_price > 0),  
  CONSTRAINT chk_book_stock CHECK (stock_qty >= 0),  
  CONSTRAINT chk_book_avail CHECK (is_available IN ('Y','N'))  
);
```

-- 3. STAFF

```
CREATE TABLE STAFF (  
  staff_id NUMBER GENERATED ALWAYS AS IDENTITY,  
  full_name VARCHAR2(150) NOT NULL,  
  email VARCHAR2(150) NOT NULL,  
  phone VARCHAR2(20),  
  role VARCHAR2(15) NOT NULL,  
  salary NUMBER(15,2) NOT NULL,  
  hire_date DATE DEFAULT SYSDATE NOT NULL,  
  CONSTRAINT pk_staff PRIMARY KEY (staff_id),  
  CONSTRAINT uq_staff_email UNIQUE (email),  
  CONSTRAINT chk_staff_role CHECK (role IN ('receptionist','cashier','manager')),  
  CONSTRAINT chk_staff_sal CHECK (salary > 0)  
);
```

-- 4. RENTAL

```
CREATE TABLE RENTAL (  
  rental_id NUMBER GENERATED ALWAYS AS IDENTITY,  
  member_id NUMBER NOT NULL,  
  book_id NUMBER NOT NULL,  
  staff_id NUMBER NOT NULL,  
  rental_date DATE DEFAULT SYSDATE NOT NULL,  
  expected_return_date DATE NOT NULL,  
  actual_return_date DATE,  
  rental_days NUMBER(5) NOT NULL,  
  rental_fee NUMBER(15,2) DEFAULT 0 NOT NULL,  
  late_fee NUMBER(15,2) DEFAULT 0 NOT NULL,  
  rental_status VARCHAR2(10) DEFAULT 'active' NOT NULL,  
  CONSTRAINT pk_rental PRIMARY KEY (rental_id),  
  CONSTRAINT fk_rental_member FOREIGN KEY (member_id) REFERENCES MEMBER(member_id),  
  CONSTRAINT fk_rental_book FOREIGN KEY (book_id) REFERENCES BOOK(book_id),
```

```
CONSTRAINT fk_rental_staff FOREIGN KEY (staff_id) REFERENCES STAFF(staff_id),
CONSTRAINT chk_rental_days CHECK (rental_days >= 1),
CONSTRAINT chk_rental_fee CHECK (rental_fee >= 0),
CONSTRAINT chk_late_fee CHECK (late_fee >= 0),
CONSTRAINT chk_rental_status CHECK (rental_status IN ('active','returned','overdue'))
);

-- 5. PAYMENT
CREATE TABLE PAYMENT (
    payment_id NUMBER GENERATED ALWAYS AS IDENTITY,
    rental_id NUMBER NOT NULL,
    staff_id NUMBER NOT NULL,
    payment_date DATE DEFAULT SYSDATE NOT NULL,
    amount_paid NUMBER(15,2) NOT NULL,
    payment_method VARCHAR2(10) NOT NULL,
    change_given NUMBER(15,2) DEFAULT 0,
    CONSTRAINT pk_payment PRIMARY KEY (payment_id),
    CONSTRAINT fk_payment_rental FOREIGN KEY (rental_id) REFERENCES RENTAL(rental_id),
    CONSTRAINT fk_payment_staff FOREIGN KEY (staff_id) REFERENCES STAFF(staff_id),
    CONSTRAINT uq_payment_rental UNIQUE (rental_id),
    CONSTRAINT chk_pay_method CHECK (payment_method IN ('cash','transfer','gopay')),
    CONSTRAINT chk_pay_amount CHECK (amount_paid > 0)
);
```

Q2 - PART A

```
-- GRANT STATEMENTS BUKU PINTAR BOOK RENTAL

-- 1. RECEPTIONIST USER
GRANT SELECT ON MEMBER TO receptionist_user;
GRANT INSERT ON MEMBER TO receptionist_user;
GRANT SELECT ON BOOK TO receptionist_user;
GRANT SELECT, INSERT ON RENTAL TO receptionist_user;
GRANT UPDATE (stock_qty) ON BOOK TO receptionist_user;

-- 2. CASHIER USER
GRANT SELECT ON RENTAL TO cashier_user;
GRANT SELECT ON MEMBER TO cashier_user;
GRANT SELECT, INSERT ON PAYMENT TO cashier_user;
GRANT UPDATE (rental_status, actual_return_date, late_fee)
    ON RENTAL TO cashier_user;

-- 3. REPORT USER (read-only on all 5 tables)
GRANT SELECT ON MEMBER TO report_user;
GRANT SELECT ON BOOK TO report_user;
GRANT SELECT ON STAFF TO report_user;
GRANT SELECT ON RENTAL TO report_user;
GRANT SELECT ON PAYMENT TO report_user;
```

Q2 - PART B

TCL: Farhan Maulana rents and returns a book

```
-- STEP 1: Insert RENTAL record
INSERT INTO RENTAL (
    member_id, book_id, staff_id,
    rental_date, expected_return_date,
    rental_days, rental_fee, late_fee, rental_status
)
VALUES (
    1, 3, 1,
    SYSDATE, SYSDATE + 5,
    5, 25000, 0, 'active'
);

-- STEP 2: Decrease book stock by 1 (book is now rented out)
UPDATE BOOK
```

```
SET stock_qty = stock_qty - 1
WHERE book_id = 3;

-- STEP 3: SAVEPOINT after rental is recorded
SAVEPOINT after_rental;
-- If payment fails later, we roll back to here
-- Rental record is kept; payment can be retried

-- STEP 4: Record the actual return + late fee
UPDATE RENTAL
SET actual_return_date = SYSDATE,
    late_fee = 4000,
    rental_status = 'returned'
WHERE rental_id = (SELECT MAX(rental_id) FROM RENTAL WHERE member_id = 1);

-- STEP 5: Insert PAYMENT record
-- Total = rental_fee + late_fee = 25000 + 4000 = 29000
-- Farhan pays 60000 cash, change = 31000
INSERT INTO PAYMENT (
    rental_id, staff_id, payment_date,
    amount_paid, payment_method, change_given
)
VALUES (
    (SELECT MAX(rental_id) FROM RENTAL WHERE member_id = 1),
    2, SYSDATE,
    60000, 'cash', 31000
);

-- STEP 6: Book is returned increase stock back by 1
UPDATE BOOK
SET stock_qty = stock_qty + 1
WHERE book_id = 3;

-- STEP 7: Everything successful COMMIT
COMMIT;

ROLLBACK SCENARIO
If PAYMENT INSERT (Step 5) fails (e.g., system error):
ROLLBACK TO SAVEPOINT after_rental;
This undoes: RENTAL update + PAYMENT insert + BOOK stock increase
This keeps: Original RENTAL record with status 'active'
Cashier can retry the return and payment process later
Full cancel if customer refuses to pay:
ROLLBACK; undoes everything including the rental
```

Q3 - PART A

```
INSERT PER TABLE in dependency order
--1. MEMBER
INSERT INTO MEMBER (member_code, full_name, phone, email, member_type, join_date, is_active)
VALUES ('MBR-2024-001', 'Farhan Maulana', '085712345678',
    'farhan.maulana@student.undip.ac.id',
    'student', TO_DATE('2024-02-10', 'YYYY-MM-DD'), 'Y');

--2. BOOK
INSERT INTO BOOK (isbn, title, author, genre, daily_price, stock_qty, is_available)
VALUES ('978-602-1234-56-7',
    'Algoritma dan Pemrograman',
    'Dr. Rinaldi Munir',
    'Academic', 5000, 3, 'Y');

--3. STAFF
INSERT INTO STAFF (full_name, email, phone, role, salary, hire_date)
VALUES ('Sari Kusumawati',
    'sari.kusuma@bukupintar.id',
    '081234567890',
    'receptionist', 3500000,
    TO_DATE('2021-03-01', 'YYYY-MM-DD'));
```

```
--4. RENTAL (rental_fee = 5 days x 5000 = 25000)
INSERT INTO RENTAL (
    member_id, book_id, staff_id,
    rental_date, expected_return_date,
    rental_days, rental_fee, late_fee, rental_status
)
VALUES (1, 1, 1,
        SYSDATE, SYSDATE + 5,
        5, 25000, 0, 'active');

--5. PAYMENT (pays 30000 cash, change = 30000 - 25000 = 5000)
INSERT INTO PAYMENT (
    rental_id, staff_id, payment_date,
    amount_paid, payment_method, change_given
)
VALUES (1, 1, SYSDATE, 30000, 'cash', 5000);

COMMIT;
```

Q3 - PART B

```
--1. BOOK: Mark unavailable if stock = 0
UPDATE BOOK
SET is_available = 'N'
WHERE stock_qty = 0;

-- 2. MEMBER: Deactivate member with overdue book > 30 days
UPDATE MEMBER
SET is_active = 'N'
WHERE member_id IN (
    SELECT member_id FROM RENTAL
    WHERE rental_status = 'active'
    AND SYSDATE - rental_date > 30
);

-- 3. RENTAL: Mark overdue for all active rentals past expected return
UPDATE RENTAL
SET rental_status = 'overdue'
WHERE rental_status = 'active'
AND expected_return_date < SYSDATE;

-- 4. STAFF: 10% salary raise for all cashiers
UPDATE STAFF
SET salary = salary * 1.10
WHERE role = 'cashier';

-- 5. BOOK: Increase price of specific book by Rp 1,000
UPDATE BOOK
SET daily_price = daily_price + 1000
WHERE title = 'Algoritma dan Pemrograman';

COMMIT;
```

Q4 - PART A

-- CASE 1: Returned/overdue rentals with total charge > 15000

```
SELECT
  m.full_name          AS "Member Name",
  m.member_type        AS "Member Type",
  b.title              AS "Book Title",
  b.genre              AS "Genre",
  s.full_name          AS "Staff Name",
  r.rental_fee + r.late_fee AS "Total Charge (Rp)",
  CASE
    WHEN m.member_type = 'student' THEN 'Yes - 10%'
    ELSE 'No'
  END                 AS "Member Discount"
FROM RENTAL r
JOIN MEMBER m ON r.member_id = m.member_id
JOIN BOOK b ON r.book_id = b.book_id
JOIN STAFF s ON r.staff_id = s.staff_id
LEFT JOIN PAYMENT p ON r.rental_id = p.rental_id
WHERE r.rental_status IN ('returned','overdue')
AND (r.rental_fee + r.late_fee) > 15000
ORDER BY (r.rental_fee + r.late_fee) DESC,
         m.full_name ASC;
```

-- CASE 2: Available books - price NOT between 3000 and 8000 OR genre Academic

```
SELECT
  b.title      AS "Book Title",
  b.author     AS "Author",
  b.genre      AS "Genre",
  b.daily_price AS "Daily Price (Rp)",
  b.stock_qty  AS "Stock",
  b.daily_price * 7 AS "Weekly Rental Cost (Rp)",
  CASE
    WHEN b.daily_price < 3000 THEN 'Budget'
    WHEN b.daily_price BETWEEN 3000 AND 8000 THEN 'Standard'
    ELSE 'Premium'
  END         AS "Price Level"
FROM BOOK b
WHERE b.is_available = 'Y'
AND (
  b.daily_price NOT BETWEEN 3000 AND 8000
  OR b.genre = 'Academic'
)
ORDER BY b.genre ASC,
         b.daily_price DESC;
```

-- CASE 3: Active/overdue student rentals > 7 days, fee above staff average

```
SELECT
  m.full_name          AS "Member Name",
  m.member_code        AS "Member Code",
  b.title              AS "Book Title",
  s.full_name          AS "Staff Name",
  TO_CHAR(r.rental_date, 'DD-MON-YYYY') AS "Rental Date",
  ROUND(SYSDATE - r.rental_date, 0)     AS "Days Rented",
  r.rental_fee          AS "Rental Fee (Rp)",
  ROUND(r.rental_fee - (
    SELECT AVG(r2.rental_fee)
    FROM RENTAL r2
    WHERE r2.staff_id = r.staff_id
  ), 0)                 AS "Above Staff Average (Rp)"
FROM RENTAL r
JOIN MEMBER m ON r.member_id = m.member_id
JOIN BOOK b ON r.book_id = b.book_id
JOIN STAFF s ON r.staff_id = s.staff_id
WHERE r.rental_status IN ('active','overdue')
AND m.member_type = 'student'
AND (SYSDATE - r.rental_date) > 7
```

```
AND r.rental_fee > (  
    SELECT AVG(r3.rental_fee)  
    FROM RENTAL r3  
    WHERE r3.staff_id = r.staff_id  
)  
ORDER BY (SYSDATE - r.rental_date) DESC;
```

